GROUP 20 ASSIGNMENT SUBMISSION

NAME	REGISTRATION NO.
NAKITENDE BUSHILLAH	26/U/04033/PS
OKALEBO ALVIN JOSEPH	26/U/0315
MUNGOMA ISAAC TREVOR	26/U/03574/PS
ANKUNDA JOSEPH BETONDE	26/U/03486/PS
WOZEI IVAN DAVID	26/U/03671/PS

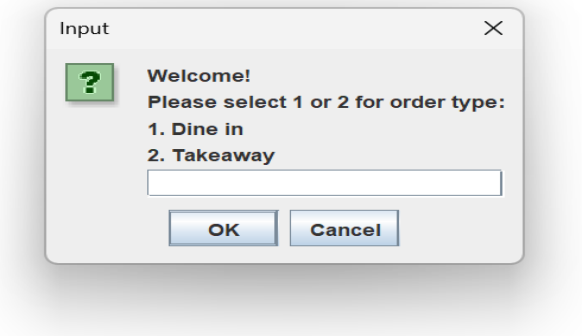
Description of Restaurant ordering System

It is a very simple and basic meal ordering system where the user is able to select a meal and the amount or quantity they want to order.

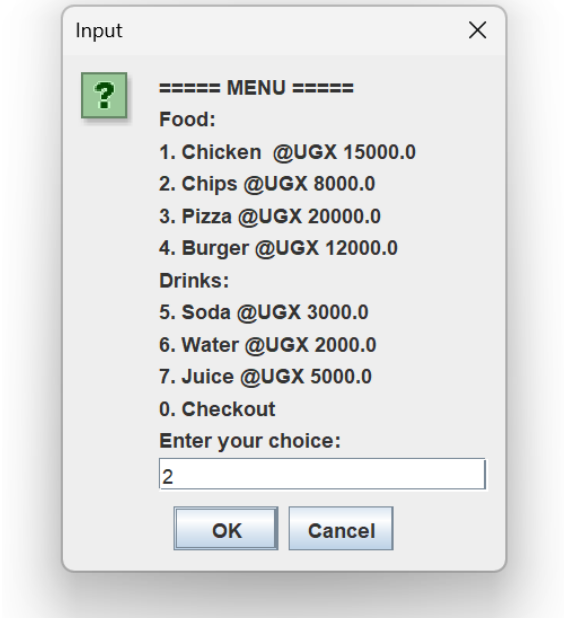
Github Link; here

Description and Workflow

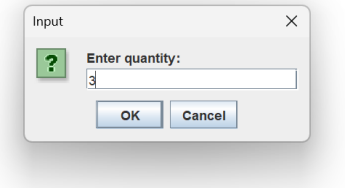
When the user selects the RestaurantOrderingSystem_Group_20.jar file, the opening tab opens which asks the user whether the meal they want to order is Dine in or Takeaway using 1 or 2 to choose respectively.



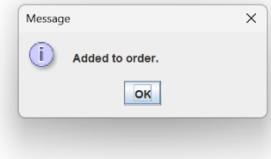
After selecting an option, the next window displaying the menu of items that can be ordered with a number that is used to select the corresponding item.



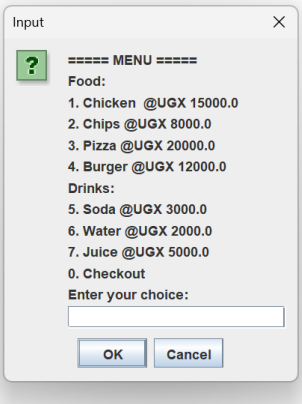
When the user selects a number, a new window asking for the quantity of items to be ordered pops up.



then

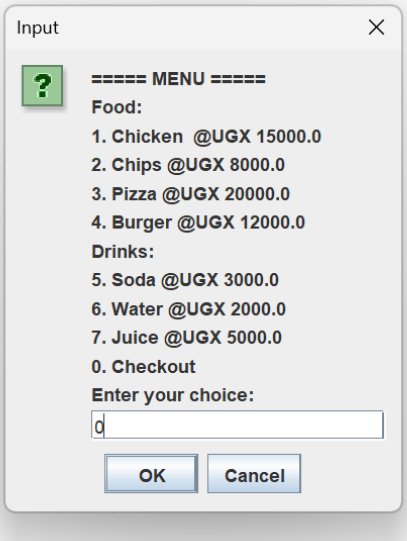


then

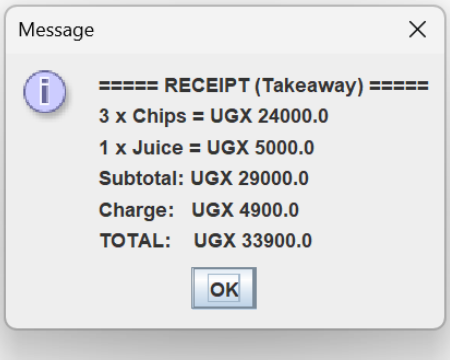


After entering in the quantity, the previous window showing the menu comes back and the user is able to select a second item.

The number zero is used to select the checkout option. A new window showing the receipt of items ordered then pops up and the system closes.



then



Main Classes

The Item class contains the blueprint of how the menu items are supposed to be that is to say the attributes. It has getter methods to access the attributes that are hidden using the private keyword. It has **subclasses namely drinks and food** that

use the same constructor as the item parent class. In the item.java file, we created the arrays of objects for each subclass each with a name and price using their respective constructors.

The Menu class mainly has methods. **printItem** that accepts an integer for number and objects to obtain its variables, as parameters, used to print items. **displayMenu** method is a for loop that uses the **printItem** method to print all the objects in the array both for the food and drink subclasses. The **get** method accepts a number as a parameter and returns a corresponding object from the arrays of either the food or drinks subclass accordingly.

The OrderItem class is a small class that has attributes for the order details but its main purpose is to copy the details of the Item class objects using a copy constructor then add on the quantity attribute. It also has a **getTotalpriceforitem** method that calculates that multiplies quantity and unit price (which is set using the **getPrice** getter method in the item class).

We also created a blank array of 15 slots that will be filled one slot at a time each time an order is made

The order class also has methods. The **addItems** method calls the **orderitem** copy constructor and stores them in the blank array for **orderitems** one at a time each time it's called of which it has a maximum of 15. It also has a counting function that helps give the limit in further looping methods to avoid looping through blank slots in the array that would cause an error. The **isEmpty** method returns a Boolean value for if the number of items ordered is zero which is applied later using the counter function. The **subtotal** metod then loops through the order array adding the total price for each item using the **getTotalpriceforitem method**.The **printreceipt** method also loops through the order array displaying the relevant order details or attributes to be seen by the user. The order class also has abstract methods **getordertype** and **getservicecharge** that are implemented by the **dineinorder and takeawayorder subclasses** to print the order type in the receipt and also calculate the service fee respectively depending on what the user wants through the use of upcasting.

The RestaurantOrderingSystem_Group_20 class is the main class where the code actually runs. It combines and uses the methods created in the previous classes to create the actual ordering system. It is also where Java Swing is

imported and implemented to create the GUI. Through the use of a while loop, we are able to ensure only valid responses are accepted and also the user is able to make a second order. Selecting 0 breaks the loop and moves on to printing the receipt., the input choice of dine in or takeaway is asked, the choice decides whether the its a dineinorder object or takeawayorder object is created and upcasted to the order class using the switch function. This is helpful when the printreceipt method is called and when calculating the subtotal. It then displays the menu and the user selects both the number corresponding to the item they want and the quantity in a nested while loop inside the while loop asking for the menu selection. The major while loop is terminated only when the user inputs zero to checkout then the receipt window displays showing the selected items and the totals after implementing the methods. The close button then terminates the system.